In [74]:
```
import pandas as pd
import numpy as np
```

Obsługa **indeksowania hierarchicznego** jest ważnym elementem biblioteki pandas umożliwiającym przypisanie do jednej osi wielu poziomów indeksowania (przypisanie dwóch lub więcej indeksów).

In [75]:
```
data = pd.Series(np.random.uniform(size=9),
                 index=[["a", "a", "a", "b", "b", "c", "c", "d", "d"],
                        [1, 2, 3, 1, 3, 1, 2, 2, 3]])
```

In [76]:
```
data
```

Out[76]:

| | | 0 |
|---|---|---|
| **a** | **1** | 0.763606 |
| | **2** | 0.208548 |
| | **3** | 0.396111 |
| **b** | **1** | 0.105956 |
| | **3** | 0.458709 |
| **c** | **1** | 0.448427 |
| | **2** | 0.447343 |
| **d** | **2** | 0.688619 |
| | **3** | 0.932637 |

**dtype:** float64

Wyświetloną czytelnie serią, której indeksem jest obiekt MultiIndex.

In [77]:
```
data.index
```

Out[77]:
```
MultiIndex([('a', 1),
            ('a', 2),
            ('a', 3),
            ('b', 1),
            ('b', 3),
            ('c', 1),
            ('c', 2),
            ('d', 2),
            ('d', 3)],
           )
```

Obiekty o indeksie hierarchicznym obsługują tzw. **indeksowanie częściowe**. Indeksowanie to umożliwia zwięzły wybór podzbioru danych

In [78]:
```
data["b"]
```

Out[78]:

| | 0 |
|---|---|
| **1** | 0.105956 |
| **3** | 0.458709 |

**dtype:** float64

In [79]:
```
data["b":"c"]
```

Out[79]:

| | | 0 |
|---|---|---|
| **b** | **1** | 0.105956 |
| | **3** | 0.458709 |
| **c** | **1** | 0.448427 |
| | **2** | 0.447343 |

**dtype:** float64

In [80]:
```
data.loc[["b", "d"]]
```

Out[80]:

| | | 0 |
|---|---|---|
| **b** | **1** | 0.105956 |
| | **3** | 0.458709 |
| **d** | **2** | 0.688619 |
| | **3** | 0.932637 |

**dtype:** float64

# Przykład na danych

In [81]:
```
# Dane sprzedażowe w dwóch miastach, w dwóch latach
df = pd.DataFrame({
    "miasto":   ["Łódź", "Łódź", "Kraków", "Kraków"],
    "rok":      [2023,   2024,   2023,     2024],
    "sprzedaz": [120,    145,    200,      230]
})
df
```

Out[81]:

| | miasto | rok | sprzedaz |
|---|---|---|---|
| **0** | Łódź | 2023 | 120 |
| **1** | Łódź | 2024 | 145 |
| **2** | Kraków | 2023 | 200 |
| **3** | Kraków | 2024 | 230 |

In [82]:
```
df.index
```

Out[82]:
```
RangeIndex(start=0, stop=4, step=1)
```

## Zwykły filtrowanie

Za każdym razem: filtr po mieście, filtr po roku, wybór kolumny. Trzy operacje dla jednej wartości.

In [83]:
```
# Sprzedaż w łodzi w 2024 - działa, ale rozwlekle:
df[(df.miasto == "Łódź") & (df.rok == 2024)]["sprzedaz"]
```

Out[83]:

| | sprzedaz |
|---|---|
| **1** | 145 |

**dtype:** int64

In [84]:
```
# Sprzedaż w Krakowie w 2023 - znowu to samo:
df[(df.miasto == "Kraków") & (df.rok == 2023)]["sprzedaz"]
```

Out[84]:

| | sprzedaz |
|---|---|
| **2** | 200 |

**dtype:** int64

## MultiIndex - indeks jako "ścieżka"

In [85]:
```
# Ten sam zbiór, ale z hierarchicznym indeksem:
sprzedaz = pd.Series(
    [120, 145, 200, 230],
    index=[["Łódź", "Łódź", "Kraków", "Kraków"],
           [2023,   2024,   2023,     2024]]
)
print(sprzedaz)
```

```
Łódź    2023    120
        2024    145
Kraków  2023    200
        2024    230
dtype: int64
```

In [86]:
```
# Sam indeks to obiekt MultiIndex:
sprzedaz.index
```

Out[86]:
```
MultiIndex([(  'Łódź', 2023),
            (  'Łódź', 2024),
            ('Kraków', 2023),
            ('Kraków', 2024)],
           )
```

## Indeksowanie - podstawowe wzorce

In [87]:
```
# Pojedyncza wartość - krotka (miasto, rok):
sprzedaz["Łódź", 2024]
```

Out[87]:
```
np.int64(145)
```

In [88]:
```
# Wszystko dla łodzi (wybór zewnętrznego poziomu):
sprzedaz["Łódź"]
```

Out[88]:

| | 0 |
|---|---|
| **2023** | 120 |
| **2024** | 145 |

**dtype:** int64

In [89]:
```
# Wszystkie miasta, ale tylko rok 2023 (wewnętrzny poziom):
sprzedaz.loc[:, 2023]
```

Out[89]:

| | 0 |
|---|---|
| **Łódź** | 120 |
| **Kraków** | 200 |

**dtype:** int64

In [90]:
```
# Zakres miast (uwaga: wymaga posortowanego indeksu):
sprzedaz.sort_index().loc["Kraków":"Łódź"]
```

Out[90]:

| | | 0 |
|---|---|---|
| **Kraków** | **2023** | 200 |
| | **2024** | 230 |
| **Łódź** | **2023** | 120 |
| | **2024** | 145 |

**dtype:** int64

# Z Series do DataFrame i z powrotem

In [91]:
```
print(sprzedaz)
```

```
łódź    2023    120
        2024    145
Kraków  2023    200
        2024    230
dtype: int64
```

In [92]:
```
# unstack - "wypchnięcie" wewnętrznego poziomu do kolumn:
tabela = sprzedaz.unstack()
print(tabela)
print(type(tabela))
```

```
        2023  2024
Kraków   200   230
łódź     120   145
<class 'pandas.core.frame.DataFrame'>
```

In [93]:
```
# stack - operacja odwrotna: kolumny wracają do indeksu:
tab_stack = tabela.stack()
print(tab_stack)
print(type(tab_stack))
```

```
Kraków  2023    200
        2024    230
łódź    2023    120
        2024    145
dtype: int64
<class 'pandas.core.series.Series'>
```

In [94]:
```
print(df)
```

```
   miasto   rok  sprzedaz
0    łódź  2023       120
1    łódź  2024       145
2  Kraków  2023       200
3  Kraków  2024       230
```

In [95]:
```
# Konwersja płaskiej ramki na hierarchiczną - set_index:
df_hier = df.set_index(["miasto", "rok"])
print(df_hier)
print(df_hier.index)
```

```
             sprzedaz
miasto rok
łódź   2023       120
       2024       145
Kraków 2023       200
       2024       230
MultiIndex([(  'łódź', 2023),
            (  'łódź', 2024),
            ('Kraków', 2023),
            ('Kraków', 2024)],
           names=['miasto', 'rok'])
```

In [96]:
```
# I z powrotem - reset_index:
df2 = df_hier.reset_index()
print(df2)
```

```
   miasto   rok  sprzedaz
0    Łódź  2023       120
1    Łódź  2024       145
2  Kraków  2023       200
3  Kraków  2024       230
```

## Tabele przestawne (ang. *Pivot Table*)

In [97]:
```
# Przekształcamy płaską ramkę w tabelę przestawną
df2_my_pivot = df.set_index(["miasto", "rok"])["sprzedaz"].unstack()
print(df2_my_pivot)
```

```
rok     2023  2024
miasto
Kraków   200   230
Łódź     120   145
```

In [98]:
```
df_pivot = pd.pivot_table(df,
                          index="miasto",
                          columns="rok",
                          values="sprzedaz")
print(df_pivot)
```

```
rok      2023   2024
miasto
Kraków  200.0  230.0
Łódź    120.0  145.0
```

In [99]:
```
df_kwartaly = pd.DataFrame({
    "miasto":   ["Łódź"]*4 + ["Kraków"]*4,
    "rok":      [2023]*4 + [2023]*4,
    "kwartal":  ["Q1","Q2","Q3","Q4"] * 2,
    "sprzedaz": [30, 25, 35, 30, 50, 45, 55, 50]
})

df_kwartaly_sum = pd.pivot_table(df_kwartaly,
                                 index="miasto",
                                 columns="rok",
                                 values="sprzedaz",
                                 aggfunc="sum")
print(df_kwartaly_sum)
```

```
rok     2023
miasto
Kraków   200
Łódź     120
```

## Agregacje po poziomie

In [100…
```
sprzedaz.index.names = ["miasto", "rok"]
sprzedaz.groupby(level="miasto").sum()
```

Out[100…

| miasto | 0 |
|---|---|
| **Kraków** | 430 |
| **Łódź** | 265 |

**dtype:** int64

In [101…

```
sprzedaz.groupby(level="rok").mean()
```

Out[101…

| rok | 0 |
|---|---|
| **2023** | 160.0 |
| **2024** | 187.5 |

**dtype:** float64

## Agregacja w DataFrame z MultiIndex

In [102…

```
frame = pd.DataFrame({
    "sprzedaz": [120, 145, 200, 230],
    "koszty":   [80,  90,  130, 140]
}, index=[["Łódź","Łódź","Kraków","Kraków"],
          [2023,  2024,  2023,    2024]])
frame.index.names = ["miasto", "rok"]
frame.groupby(level="miasto").sum()
```

Out[102…

| miasto | sprzedaz | koszty |
|---|---|---|
| **Kraków** | 430 | 270 |
| **Łódź** | 265 | 170 |

In [103…

```
frame.groupby(level="miasto").agg(["sum", "mean"])
```

Out[103…

| | sprzedaz | | koszty | |
|---|---|---|---|---|
| miasto | sum | mean | sum | mean |
| **Kraków** | 430 | 215.0 | 270 | 135.0 |
| **Łódź** | 265 | 132.5 | 170 | 85.0 |

In [104…

```
frame["sprzedaz"].groupby(level="miasto").agg(
    total="sum", srednia="mean", rozstep=lambda x: x.max()-x.min()
    )
```

Out[104…

| miasto | total | srednia | rozstep |
|---|---|---|---|
| **Kraków** | 430 | 215.0 | 30 |
| **Łódź** | 265 | 132.5 | 25 |

# *Zadania*

Sieć "ModaŁódź" ma sklepy w trzech miastach. Każdy sklep sprzedaje trzy kategorie produktów. Dane obejmują 4 kwartały 2023 i 2024 roku.

In [105…

```
print(df.columns)
print(df.head())
```

```
Index(['miasto', 'rok', 'sprzedaz'], dtype='object')
   miasto   rok  sprzedaz
0    Łódź  2023       120
1    Łódź  2024       145
2  Kraków  2023       200
3  Kraków  2024       230
```

## Zadanie 1 — Budowanie MultiIndex

a) Utwórz ramkę `df_hi` z hierarchicznym indeksem ( `miasto` , `kategoria` , `rok` , `kwartal` ). Posortuj indeks.

b) Wyświetl liczbę poziomów indeksu i ich nazwy.

c) Ile unikalnych kombinacji indeksu istnieje? Użyj `.index` do odpowiedzi.

In [113…

```
import pandas as pd
import numpy as np

miasta = ["Łódź", "Kraków", "Warszawa"]
kategorie = ["Damska", "Męska", "Dziecięca"]
lata = [2023, 2024]
kwartaly = ["Q1", "Q2", "Q3", "Q4"]

idx = pd.MultiIndex.from_product([miasta, kategorie, lata, kwartaly], names=["mi
df = pd.DataFrame(index=idx).reset_index()

np.random.seed(42)
df["przychod_tys"] = np.random.randint(50, 200, size=len(df))
df["koszt_tys"] = np.random.randint(30, 150, size=len(df))

df_hi = df.set_index(["miasto", "kategoria", "rok", "kwartal"]).sort_index()

print(df_hi)
print(df_hi.index.nlevels)
print(df_hi.index.names)

print(len(df_hi.index.unique()))
```

```
                                przychod_tys  koszt_tys
miasto kategoria rok  kwartal
Kraków Damska    2023 Q1                 98        132
                      Q2                108         70
                      Q3                 64         57
                      Q4                100         36
                 2024 Q1                157        102
...                                     ...        ...
Łódź   Męska     2023 Q4                149        137
                 2024 Q1                153         37
                      Q2                180        117
                      Q3                199         92
                      Q4                102         40

[72 rows x 2 columns]
4
['miasto', 'kategoria', 'rok', 'kwartal']
72
```

# Zadanie 2 — Selekcje na MultiIndex

a) Wybierz wszystkie dane dla Łodzi.

b) Wybierz dane dla Łodzi, kategorii Damska.

c) Wybierz dane dla wszystkich miast, ale tylko rok 2024. (Wskazówka: użyj metodę `xs()` )

d) Wybierz Q4 z obu lat, ale tylko dla Krakowa i kategorii Męska.

In [121…

```
dane_lodz = df_hi.loc["Łódź"]

dane_lodz_damska = df_hi.loc[("Łódź", "Damska")]

dane_2024 = df_hi.xs(2024, level="rok")

dane_krakow_meska_q4 = df_hi.loc[("Kraków", "Męska", slice(None), "Q4"), :]

print(dane_lodz)
print("\n--- ### ---\n")
print(dane_lodz_damska)
print("\n--- ### ---\n")
print(dane_2024)
print("\n--- ### ---\n")
print(dane_krakow_meska_q4)
```

```
                        przychod_tys  koszt_tys
kategoria rok  kwartal
Damska    2023 Q1                152        138
               Q2                142         70
               Q3                 64         58
               Q4                156         44
          2024 Q1                121         74
               Q2                 70         94
               Q3                152        118
               Q4                171        100
Dziecięca 2023 Q1                 51        144
               Q2                137        110
               Q3                 87         37
               Q4                179         64
          2024 Q1                 70         64
               Q2                107         62
               Q3                 71         34
               Q4                138        135
Męska     2023 Q1                124         38
               Q2                137        117
               Q3                166         30
               Q4                149        137
          2024 Q1                153         37
               Q2                180        117
               Q3                199         92
               Q4                102         40

--- ### ---

              przychod_tys  koszt_tys
rok  kwartal
2023 Q1                152        138
     Q2                142         70
     Q3                 64         58
     Q4                156         44
2024 Q1                121         74
     Q2                 70         94
     Q3                152        118
     Q4                171        100

--- ### ---

                            przychod_tys  koszt_tys
miasto   kategoria kwartal
Kraków   Damska    Q1                157        102
                   Q2                104        101
                   Q3                113         41
                   Q4                180         63
         Dziecięca Q1                179         64
                   Q2                133         94
                   Q3                141        128
                   Q4                160        130
         Męska     Q1                 67         91
                   Q2                181        117
                   Q3                138         66
                   Q4                109        128
Warszawa Damska    Q1                153         34
                   Q2                181        119
                   Q3                 51        148
                   Q4                183         43
```

```
         Dziecięca Q1                160        142
                   Q2                102         81
                   Q3                 73        125
                   Q4                173         33
         Męska     Q1                195         44
                   Q2                 93        119
                   Q3                 63         71
                   Q4                144        106
Łódź     Damska    Q1                121         74
                   Q2                 70         94
                   Q3                152        118
                   Q4                171        100
         Dziecięca Q1                 70         64
                   Q2                107         62
                   Q3                 71         34
                   Q4                138        135
         Męska     Q1                153         37
                   Q2                180        117
                   Q3                199         92
                   Q4                102         40

--- ### ---

                                 przychod_tys  koszt_tys
miasto kategoria rok  kwartal
Kraków Męska     2023 Q4                 122         52
                 2024 Q4                 109        128
```

# Zadanie 3 — pivot_table: roczne podsumowanie

a) Utwórz tabelę przestawną `tab_miasta`, która pokaże **sumę przychodów** w wierszach per miasto, w kolumnach per rok.

b) Dodaj do tabeli kolumnę `zmiana_proc` — procentową zmianę przychodu między 2023 a 2024. Które miasto rosło najszybciej?

c) Utwórz drugą tabelę przestawną, w której wiersze to (miasto, kategoria), kolumny to rok, wartości to *średni przychód kwartalny*. Która kombinacja (miasto, kategoria) ma najwyższy średni przychód w 2024?

In [109…

```
tab_miasta = pd.pivot_table(df_hi.reset_index(), index="miasto", columns="rok",

tab_miasta["zmiana_proc"] = ((tab_miasta[2024] - tab_miasta[2023]) / tab_miasta[
print(tab_miasta["zmiana_proc"].idxmax())

tab_kategorie = pd.pivot_table(df_hi.reset_index(), index=["miasto", "kategoria"
print(tab_kategorie[2024].idxmax())
```

```
Kraków
('Łódź', 'Męska')
```

# Zadanie 4 — Agregacja po poziomach

a) Na `df_hi` oblicz sumę przychodów per miasto (agreguj po poziomie `"miasto"`).

b) Oblicz **średni przychód kwartalny per kategoria** (agreguj po poziomie `"kategoria"`).

c) Użyj `.agg(["sum", "mean", "max"])` na kolumnie `przychod_tys` pogrupowanej po `(miasto, rok)`. Który wiersz ma najwyższą wartość `max`?

In [110…

```
suma_per_miasto = df_hi.groupby(level="miasto")["przychod_tys"].sum()

srednia_per_kategoria = df_hi.groupby(level="kategoria")["przychod_tys"].mean()

agg_miasto_rok = df_hi.groupby(level=["miasto", "rok"])["przychod_tys"].agg(["su
print(agg_miasto_rok["max"].idxmax())
```

```
('Łódź', np.int64(2024))
```

# Zadanie 5 — Marża i ranking

a) W `df_hi` dodaj kolumnę `marza_tys = przychod_tys - koszt_tys`.

b) Utwórz tabelę przestawną: *wiersze* = *miasto*, *kolumny* = *kategoria*, *wartości* = *suma marży*. Które miasto jest najbardziej zyskowne? Która kategoria generuje najwyższą marżę?

In [111…

```
df_hi["marza_tys"] = df_hi["przychod_tys"] - df_hi["koszt_tys"]

tab_marza = pd.pivot_table(df_hi.reset_index(), index="miasto", columns="kategor

print(tab_marza.sum(axis=1).idxmax())

print(tab_marza.sum(axis=0).idxmax())
```

```
Łódź
Męska
```